数据结构基础

2026/1/16

1.1 数据结构的定义

数据结构是一种组织、管理和存储数据的方式。

不同种类的数据结构适合不同种类的应用，部分数据结构甚至是为了解决特定问题而设计出来的。正确的数据结构选择可以提高算法的性能。在计算机程序设计的过程中，选择适当的数据结构是一项重要工作。

1.2 数据结构的操作

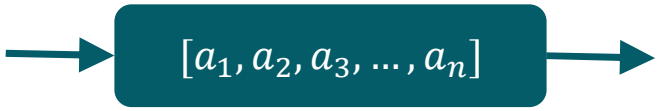
操作是封装为模块的数据结构与外部数据的交互，分为修改操作、查询操作，以维护数值序列的数据结构为例，下面罗列了部分将来会见到的常见操作，同种数据结构可能仅支持其中一部分，其余操作并不支持：

修改操作：

给序列任一位置重新赋值（单点修改）

给序列的某段同时加上一个数（区间修改）

在任一位置后面插入新元素、清空所有元素……



查询操作：

获取序列任一位置的当前值（单点查询）

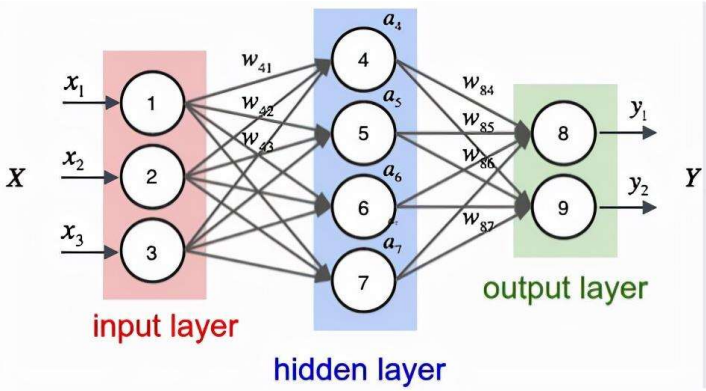
获取序列某段的最大值（区间查询）

查询全局总和、查询有多少元素等于某个值……

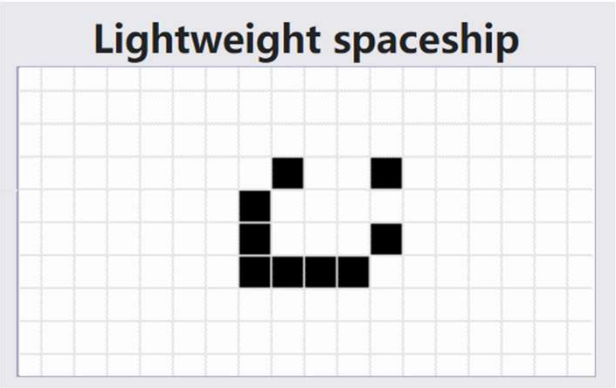
1.3 “细胞”思想

数据结构内部一般用“细胞”思想来管理数据，使得各操作的代码实现既高效、又整洁。

用“神经元”组成了网络



用“单元格”组成了康威生命游戏



用“举旗的人”组成了人列计算机

好处在于，当想将维护整数的任务变为维护小数的任务，只需统一改变“细胞”类型即可。

思考：你认识的最简单的数据结构是什么？

1.4 数组

数组是最简单的数据结构。

```
int a[100000];
double f[1000000];
```

“细胞”：

数据类型相同的数组元素

修改操作：

给序列第 i 个元素重新赋值

查询操作：

获取第 i 个元素的当前值

1.5 链表

链表是另一类最基础的数据结构。

C++ 特性：模板与泛化

```
template <typename T>
struct Node { T data; Node<T>* prev, last; };
Node<int>* head, tail;
```

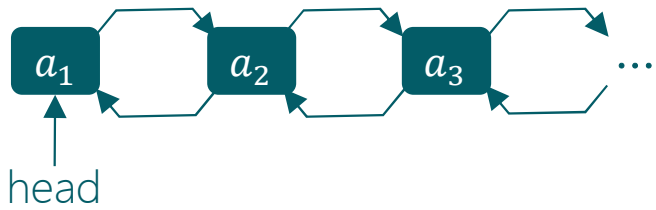
“细胞”：

数据类型相同的链表结点，需要特殊关注链表头（或链表尾）

修改操作：

给序列第 i 个元素重新赋值

删除任一元素、在任一元素前插入新元素



查询操作：

获取第 i 个元素的当前值

用指针直接访问任一元素的当前值

思考：链表也能支持数组的各种操作，是否说明链表一定可以取代数组？

1.6 复杂度

尽管同个操作可以被不同数据结构支持，但由于实现算法不同，耗费的时间、内存必然会有所差异。评价算法性能的标准便是复杂度，更具体可分为时间复杂度和空间复杂度，其中时间复杂度用于估计执行一遍算法时计算机所需的运算次数，空间复杂度用于估计执行一遍算法时计算机所需分配的内存。

```
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++)
        ans += i / j;
```

出现了 =（赋值运算）、<=（比较运算）、++（自增运算）、/（除法运算）、+=，总运算次数接近 n^2 的有限倍

```
void func(int n) {
    if (n == 0) return;
    ans += n;
    func(n / 2);
}
```

每次递归需要额外分配一个 int，总分配内存接近 $\log_2 n$

在算法竞赛中，一般假设出题人作恶多端，造最毒瘤的数据卡你的程序，所以复杂度一般优先分析最劣复杂度，然后是平均复杂度、最优复杂度。

1.6 复杂度

比较两个算法的优劣，实质上是比较数据规模趋于无限时，谁复杂度的“等价无穷大”阶数更低。例如，运算 $100n$ 次的算法，可以认为比运算 n^2 次的算法更优。运算 $n\ \log_2 n$ 次的算法，可以认为和运算 $n\ \log_{100} n$ 次的算法一样优（所以分析复杂度时遇到 $\log_x$ 时统一写 log ）。

$$1 < n < n\log n < n\log^2 n < n\sqrt{n} < n\sqrt{n\log n} < n\sqrt{n}\log n < n^2 < n^3 < 2^n < 3^n < n! < n^n$$

$f(n)$ 的复杂度阶数不高于 $g(n)$ 记作 $\boldsymbol{f(n) = O(g(n))}$，框定了一个算法的性能上界。
$f(n)$ 的复杂度阶数不低于 $g(n)$ 记作 $\boldsymbol{f(n) = \Omega(g(n))}$，框定了一个算法的性能下界。

例：
$5n2^{n-1} + 9 \cdot 2^{n+1} + 2n^3 + 7$ 是 $O(n2^n)$ 的
刚才的两层循环，时间复杂度是 $O(n^2)$ 的（复杂度未指明平均还是最优最劣默认指最劣）
如果算法要输入 n 个数，时间复杂度是 $\Omega(n)$ 的
快速排序，平均时间复杂度是 $O(n\log n)$ 的（最劣时间复杂度是 $O(n^2)$ 的）
基于比较的排序，时间复杂度是 $\Omega(n\log n)$ 的

1.6 复杂度

现在请回答：

数组任意单次修改第 i 个元素的时间复杂度

链表任意单次查询第 i 个元素的时间复杂度

链表用指针访问一个元素并在它前面插入单个新元素的时间复杂度

1.7 常数

为什么我的程序时间复杂度是 $O(n \log n)$ 的，同学是 $O(n \log^2 n)$ 的，我反而 TLE 了他没有？

答：你的常数太大了，被题目“卡常”了。

由于复杂度分析是理想情况，在实际情况中运算 $30 \cdot n \log n$ 次自然可能跑不过 $n \log^2 n$ 次，要从题目的数据范围、自己代码的优化程度具体情况具体分析，本应忽略不计的数即为常数。

当然，我们一般还是遵循复杂度为主，然后在此基础上优化常数。可以假设评测机一秒内能运算 10^8 次，一个 $O(n^2)$ 的算法理论上能跑 $n \leq 10000$，加上常数的因素可能就 $n \leq 5000$ 了。

2.1 存储型数据结构的定义

继承数组和链表，可以设计用途更针对性的数据结构。

存储型数据结构，顾名思义，以存储的用途为主，我们关心每个“细胞”（数据单元）本身何时进入数据结构的存储、何时退出数据结构的存储，不关心它们之间能产生什么额外信息，一般也就不需要更多“细胞”去维护这些信息。

2.2 栈

栈是“后进先出”的存储型数据结构。

修改操作：

push(x) - 将新的元素 x 加到栈顶

pop() - 将栈顶的元素移除

查询操作：

top() - 返回当前栈顶元素的值

2.2 栈

基于数组实现的栈：

```
int top = 0, stack[1000];
push: stack[++top] = x;
pop: top--;
top: stack[top];
```

基于链表实现的栈：

```
Node<int>* top = new Node<int>;
push: 链表的插入，略
pop: 链表的删除，略
top: top->data;
```

同：无论数组还是链表，所有操作的时间复杂度都是 $O(1)$ 的。

异：数组有开得不够大的风险，导致“爆栈”；链表实现麻烦。

2.2 栈

实际上，我们可以学会使用标准模板库（Standard Template Library，即 STL）。它提供了现有的封装好的栈，并支持更多的操作。

STL 的参考文档请翻阅 cppreference.com。

cppreference.com

Page Discussion

C++ Containers library std::stack

std::stack

Defined in header <stack>

```
template<
    class T,
    class Container = std::deque<T>
> class stack;
```

The std::stack class is a container adaptor that gives the programmer the functionality of a stack - specifically, a LIFO (last-in, first-out) data structure.

Member functions

(constructor)	constructs the stack (public member function)
(destructor)	destructs the stack (public member function)
operator=	assigns values to the container adaptor (public member function)
Element access	
top	accesses the top element (public member function)
Capacity	
empty	checks whether the container adaptor is empty (public member function)
size	returns the number of elements (public member function)
Modifiers	
push	inserts element at the top (public member function)
push_range (C++23)	inserts a range of elements at the top (public member function)
emplace (C++11)	constructs element in-place at the top (public member function)
pop	removes the top element (public member function)
swap (C++11)	swaps the contents (public member function)

```
std::stack<int> sta;
// 或 stack<int> sta 如果你 using namespace std 了的话
sta.push(x);
if (!sta.empty()) sta.pop();
```

2.3 队列

队列是“先进先出”的存储型数据结构。

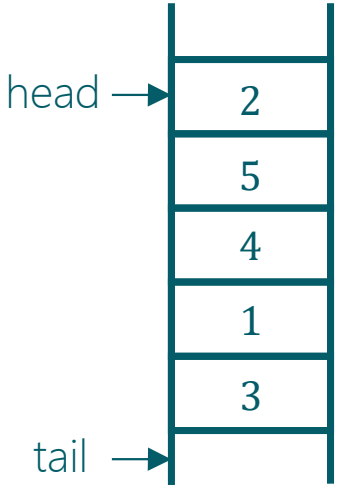
修改操作：

push(x) - 将新的元素 x 加到队尾

pop() - 将队首的元素移除

查询操作：

front() - 返回当前队首元素的值



2.3 队列

基于数组实现的队列：

```
int head = 0, tail = 0, queue[1000];
push: queue[tail++] = x;
pop: head++;
front: queue[head];
```

基于链表实现的队列：

```
Node<int>* head, tail;
push: 链表的插入，略
pop: 链表的删除，略
front: head->data;
```

同：无论数组还是链表，所有操作的时间复杂度都是 O(1) 的。

异：与栈类似，数组有开得不够大的风险；链表实现麻烦。

2.3 队列

STL 提供了现有的封装好的队列，并支持更多的操作。

cppreference.com

Page Discussion

C++ Containers library std::queue

std::queue

Defined in header <queue>

```
template<
    class T,
    class Container = std::deque<T>
> class queue;
```

The std::queue class template is a container adaptor that gives the functionality of a queue - specifically, a FIFO (first-in, first-out) data structure.

Member functions

(constructor)	constructs the queue (public member function)
(destructor)	destructs the queue (public member function)
operator=	assigns values to the container adaptor (public member function)
Element access	
front	access the first element (public member function)
back	access the last element (public member function)
Capacity	
empty	checks whether the container adaptor is empty (public member function)
size	returns the number of elements (public member function)
Modifiers	
push	inserts element at the end (public member function)
push_range (C++23)	inserts a range of elements at the end (public member function)
emplace (C++11)	constructs element in-place at the end (public member function)
pop	removes the first element (public member function)
swap (C++11)	swaps the contents (public member function)

```
std::queue<int> que;
// 或 queue<int> que 如果你 using namespace std 了的话
que.push(x);
while (que.size()) que.pop();
```

2.4 栈的应用：括号匹配

给定一个含有运算符（+ - * /）、括号（( )）、数字的表达式（以字符串形式输入），求表达式中每个右括号会和哪个左括号匹配。例如 ((5+1)-3*(8+4))/(6-2)。

暴力：每次先找最内层的括号匹配然后移除掉，既要多次扫描又要维护删除，起码 $O(n^2)$ 。

栈：只需从左到右一次扫描，遇到左括号就入栈，遇到右括号就出栈并匹配，即可 $O(n)$。

```
std::stack<int> sta;
for (int i = 1; i <= n; i++)
    if (expression[i] == ‘(’)
        sta.push(i);
    else if (expression[i] == ‘)’)
        { match[i] = sta.top(); sta.pop(); }
```

2.5 栈的应用：单调栈

给定长度为 n 的序列 $A=[a_1,a_2,\dots,a_n]$，求每个数左侧严格比它小的数中最近的下标（不存在即 0），例如：

输入 8 9 3 2 5 5 1 7 2　　输入 2 7 1 5 5 2 3 9 8

输出 0 1 0 0 4 4 0 6 6　　输出 0 1 0 3 3 3 7 7 7

暴力：每个数都往前扫，遇到严格更小的数就停止，可以被卡到 $O(n^2)$。

栈：先假设“哨兵”元素 $a_0=-\infty$ 并将其入栈，然后从左到右一次扫描，每个数都会入栈，但入栈前会消灭掉栈顶那些不小于它的元素，这些元素出栈后栈顶即是最近的更小的元素，之所以称为单调栈，是因为当前扫到的数入栈仍会保持栈“从底到顶”值的单调性。每个数入栈恰好一次，出栈至多一次，时间复杂度为 $O(n)$。

```
for (int i = 1; i <= n; i++) {
    while (a[sta.top()] >= a[i]) sta.pop();
    ans[i] = sta.top(); sta.push(i);
}
```

2.6 队列的应用：单调队列

单调队列是队列的改装，查询队首 front() 被查询队内最大最小 top() 替代，修改不变。

原理：“在集训队里，当一名选手比你小还比你强，那么你就可以退役了”。

意思是，在任意元素入队之前，检查队尾的元素是否劣于当前元素，劣则说明它们在队列里一定将来先出队、值还不占优势，不如现在就淘汰（移除）得了。任何时刻，你发现队列从队首到队尾永远像单调栈那样值具有单调性，队首的值更优但已“垂暮”、队尾的值更劣但“时日方长”，查询 top() 就相当于查询已经淘汰无用元素后的 front()。

2.6 队列的应用：单调队列

给定长度为 n 的序列 $A = [a_1, a_2, \dots, a_n]$ 和 m，求每个长度为 m 的子段的最大值。

建模：$a_1, a_2, \dots, a_m$ 依次入单调队列，查询，接下来依次入 a_i $(i > m)$、出 a_{i-m}、查询。每个数入队恰好一次，出队至多一次，时间复杂度为 $O(n)$。

因为单调队列需要支持移除队尾，这点 std::queue 不满足条件，实现上需要用到队首、队尾都能加入、移除元素的 std::deque。

```
std::deque<int> que;
for (int i = 1; i <= n; i++) {
    while (!que.empty() && que.front() <= i - m) que.pop_front();
    while (!que.empty() && a[que.back()] < a[i]) que.pop_back();
    que.push(i);
    if (i >= m) ans[i - m + 1] = a[que.front()];
}
```

2.7 优先队列

优先队列是“大值先出”的存储型数据结构。

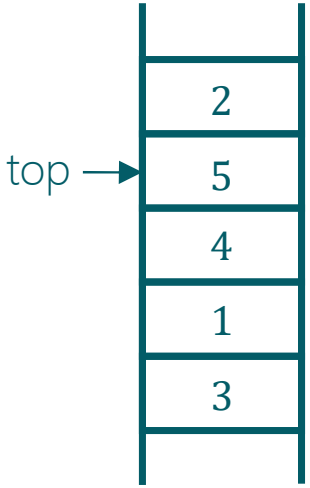
修改操作：

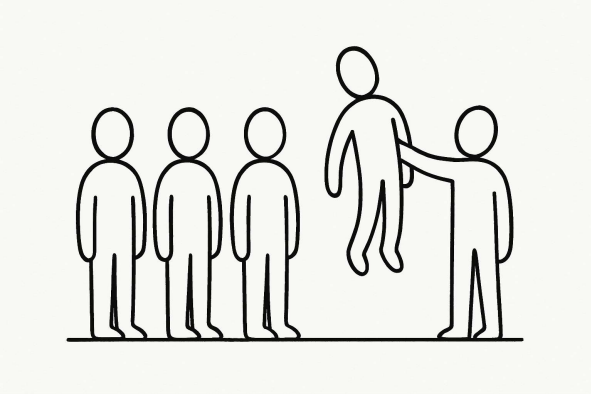
push(x) - 将新的元素 x 加入队列

pop() - 将最大元素从队列中移除

查询操作：

top() - 返回当前最大元素的值





注意和单调队列的区别，后者移除的仍然是靠前而不是最优的元素，优先队列的修改操作时间复杂度是 $O(\log)$ 而不是 O(1)（稍后我们会证明）。

2.7 优先队列

STL 提供了现有的封装好的优先队列，并支持更多的操作。

cppreference.com Search

Page Discussion

C++ Containers library std::priority_queue

std::priority_queue

Defined in header <queue>

```
template<
    class T,
    class Container = std::vector<T>,
    class Compare = std::less<typename Container::value_type>
> class priority_queue;
```

The priority queue is a container adaptor that provides constant time lookup of the largest (by default) element, at the expense of logarithmic insertion and extraction.

Member functions

(constructor)	constructs the priority_queue (public member function)
(destructor)	destructs the priority_queue (public member function)
operator=	assigns values to the container adaptor (public member function)
Element access	
top	accesses the top element (public member function)
Capacity	
empty	checks whether the container adaptor is empty (public member function)
size	returns the number of elements (public member function)
Modifiers	
push	inserts element and sorts the underlying container (public member function)
push_range (C++23)	inserts a range of elements and sorts the underlying container (public member function)
emplace (C++11)	constructs element in-place and sorts the underlying container (public member function)
pop	removes the top element (public member function)
swap (C++11)	swaps the contents (public member function)

```
std::priority_queue<int> que;
// std::priority_queue<int, std::vector<int>,
                    std::greater<int>> 定义小值先出的优先队列
que.push(x);
x = que.top();
```

2.8 优先队列的应用：维护中位数

基于优先队列，实现一个数据结构，支持：插入一个数、查询当前所有数的中位数。

建模：设当前共 m 个数，用一个“大值先出”的优先队列维护中位数以下的、较小的 $\lfloor m/2 \rfloor$个数，用一个“小值先出”的优先队列维护中位数以上的、较大的 $\lceil m/2 \rceil$个数。

在插入过程中如果优先队列的大小不平衡了，则将元素过多的优先队列的 top() 移除加入元素过少的优先队列。

然后如果“大值先出”的优先队列的 top() 大于“小值先出”的优先队列的 top()，则分别移除并加入对侧的优先队列。

该结构保证了，任意时刻中位数等于“大值先出”的优先队列的 top()。

时间复杂度为 $O(n \log n)$（n 为操作次数）

2.9 动态数组

动态数组是数组的加强版，其在保留数组用下标修改、查询的特性的基础上，支持改变数组大小、O(1) 地在末端加入或删除元素等动态操作。

STL 提供了现有的封装好的动态数组 std::vector<T>，极其常用。

std::vector

Defined in header <vector>

```
template<
    class T,
    class Allocator = std::allocator<T>
> class vector;
```

Modifiers

clear	clears the contents (public member function)
insert	inserts elements (public member function)
insert_range (C++23)	inserts a range of elements (public member function)
emplace (C++11)	constructs element in-place (public member function)
erase	erases elements (public member function)
push_back	adds an element to the end (public member function)
emplace_back (C++11)	constructs an element in-place at the end (public member function)
append_range (C++23)	adds a range of elements to the end (public member function)
pop_back	removes the last element (public member function)
resize	changes the number of elements stored (public member function)
swap	swaps the contents (public member function)

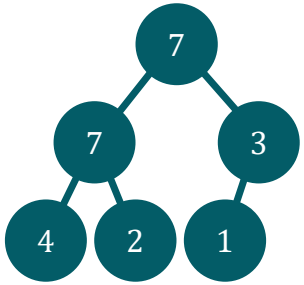
```
std::vector<int> vec(n);
for (auto& x : vec) std::cin >> x;
vec.push_back(0);
std::sort(vec.begin(), vec.end());
vec.pop_back();
vec.resize(n / 2);
```

3.1 会思考的“细胞”：堆

不同于栈、队列的线性排布，优先队列内部数据单元是呈树状排布的，具体地，它构成一个大根堆。

大根堆是一棵满足以下条件的带点权完全二叉树：

- 只有一个节点的完全二叉树是堆；
- 根节点左右儿子为空、或根权值更小的堆的完全二叉树是堆。



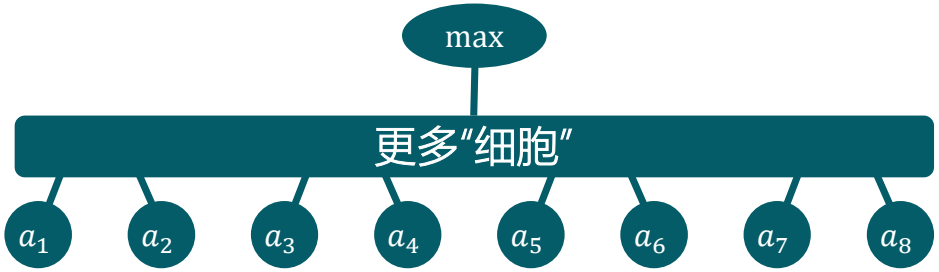
小根堆即是从浅到深权值依次增大的带点权二叉树。

思考：这样的结构为什么能使优先队列支持快速插入和删除？

3.2 设计额外“细胞”：胜者树

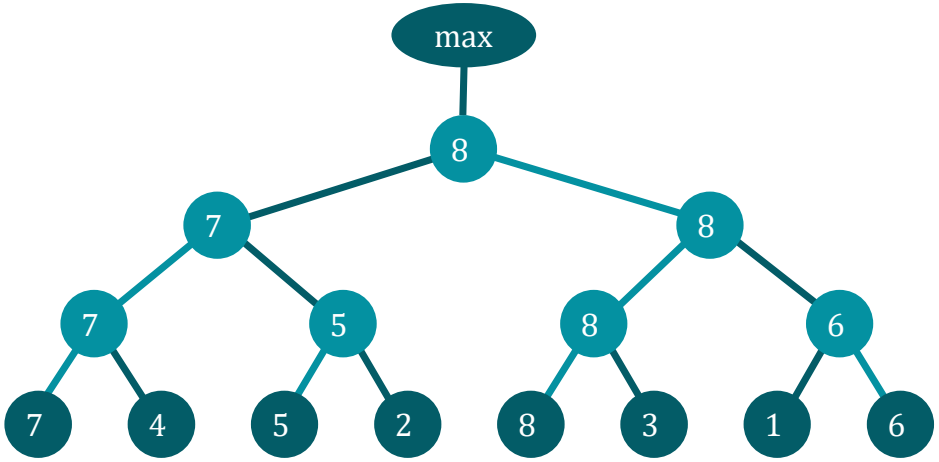
很多时候，像存储型数据结构每个“细胞”直接代表被存储量是远远不够的。

例如，你需要维护长度为 n 的序列 $A = [a_1, a_2, \ldots, a_n]$，支持单点修改、查询全局最大值。显然该问题不能用优先队列做，因为执行修改操作时你无法锁定被修改元素在堆里的位置，这就要求我们设计额外的数据单元来维护更多的信息。



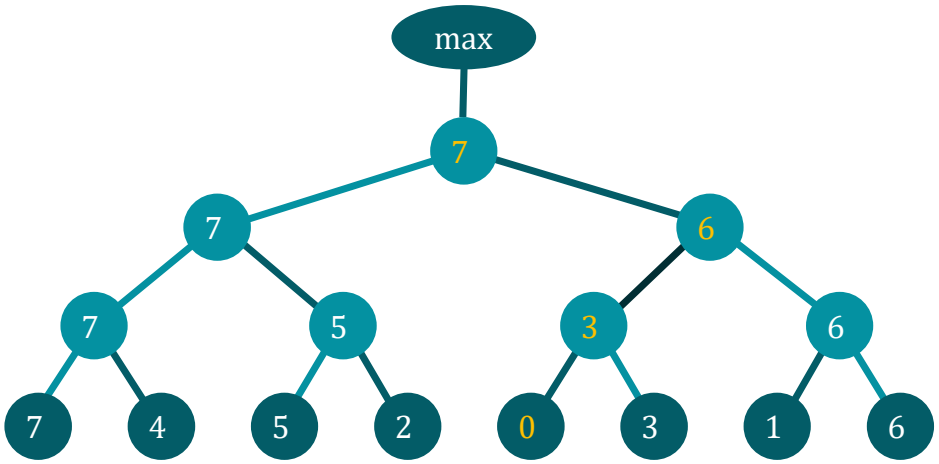
3.2 设计额外“细胞”：胜者树

怎么解决呢？不妨借鉴生活中的“16/8/4/2 强”晋级机制，让数字之间“决斗”，这样轮数远比从左往右扫、不断更新最大值要小。该二叉树结构叫作胜者树。



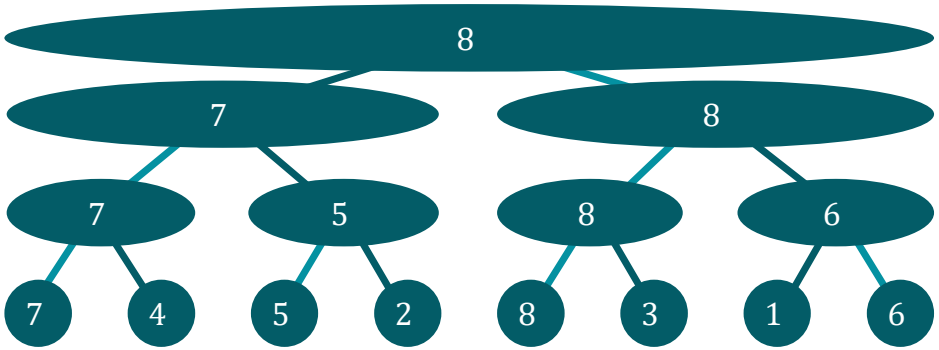
3.2 设计额外“细胞”：胜者树

在胜者树中，每个非叶“细胞”的职责是取它两个儿子中的较大值，当一次修改发生时，我们只需每层至多动一个“细胞”的信息。



3.2 设计额外“细胞”：胜者树

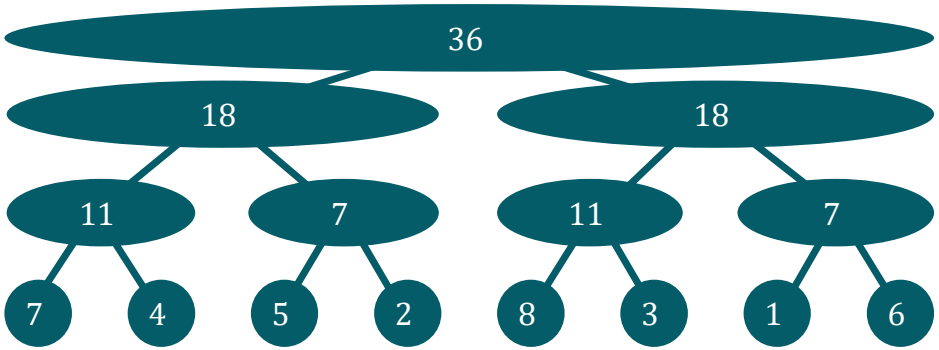
如何统一原有“细胞”和额外“细胞”所维护信息的描述呢？我们可以说，每个“细胞”维护的都是某段固定区间里的最大值。



3.2 设计额外“细胞”：胜者树

如果针对的是求和任务，那么道理是一样的，每个“细胞”维护的是区间总和。

用这种思路维护序列信息，本质上是构造出一个区间集 I，修改操作的时间复杂度即是最坏会对多少个区间的信息产生影响，查询操作的时间复杂度即是最坏要用至少多少个区间来合成出目标区间的答案。



3.3 前缀和

构造区间集 $I = \{[1,1],[1,2],\dots,[1,n]\}$。

修改，最坏每次改第一个数上，n 个区间的信息都被影响，时间复杂度 $O(n)$。

查询，最坏每次只要用 2 个区间的信息一减得出任意区间信息，时间复杂度 $O(1)$。

应用中，经常用该结构预处理完区间的信息，即前缀和，接下来便不再调用低效的修改，只调用高效的查询。

3.4 运算的性质

可不可以用该结构预处理完前缀最大值，每次 O(1) 查询区间最大值？

不可以。因为 max 运算不满足消去律，即你无法从 $\max(?, x) = x$ 中倒推 ? 是什么，自然就不能像 + 运算那样用逆运算 − 减出一个区间的信息。

同理，按位与、按位或、gcd 也不满足消去律。但它们具备加法不满足的性质，即吸收律，意思是两个重叠的区间可以合并成整个区间，但加法直接合并会算重重叠的部分。

即查询时，满足消去律的可以区间相消，满足吸收律的可以区间重叠。

思考：如何构造支持 $O(n \log n)$ 预处理 $O(1)$ 查询区间最大值的区间集？

感谢您的聆听